

Day 3: 后端开发技能提升 *>

*今日时间安排

时段	时间	内容	形式
上午	9:00-10:30	Python高级特性与异步编程	理论+实践
	10:45-12:00	FastAPI框架入门与核心概念	框架学习
下午	14:00-15:30	RESTful API设计与实现	动手编码
	15:45-17:00	数据验证、错误处理与中间件	项目实战
晚上	19:00-20:30	数据库ORM集成与CRUD操作	完整应用
	20:45-21:00	API测试与文档生成	自测

√ √ 学习目标

今日完成后，你将能够：

√ 运用Python高级特性 - 使用装饰器、上下文管理器、生成器等 √ 掌握异步编程 - 理解async/await、事件循环、并发模型 √ 构建FastAPI应用 - 路由、依赖注入、请求响应处理 √ 设计RESTful API - 遵循REST规范，设计清晰的API接口 √ 实现数据验证 - 使用Pydantic进行请求验证和序列化 √ 集成数据库ORM - 使用SQLAlchemy操作PostgreSQL

** 详细学习内容

1. Python 高级特性 (1.5小时)

1.1 装饰器 (Decorators)

基础装饰器

```
def timer(func):  
    """计算函数执行时间的装饰器"""  
    import time  
  
    def wrapper(*args, **kwargs):  
        start = time.time()  
        result = func(*args, **kwargs)  
        end = time.time()  
        print(f"{func.__name__} * 执行耗时: {end - start:.4f*秒}")  
        return result  
  
    return wrapper
```

@timer

```
def slow_function():  
    import time  
    time.sleep(1)  
    return "完成"
```

带参数的装饰器

```
def repeat(times=3):  
    """重复执行函数多次的装饰器"""  
    def decorator(func):  
        def wrapper(*args, **kwargs):  
            results = []  
            for _ in range(times):  
                result = func(*args, **kwargs)  
                results.append(result)  
            return results  
        return wrapper  
    return decorator
```

@repeat(times=5)

```
def greet(name):  
    print(f"Hello, {name*!}")  
    return name
```

类作为装饰器

```
class Logger:  
    """日志记录装饰器类"""  
  
    def __init__(self, prefix=""):  
        self.prefix = prefix
```

```
def __call__(self, func):
```

```
def wrapper(*args, **kwargs):
```

```
print(f"[{self.prefix*} 调用 {func.__name__*}"])
```

try:

```
result = func(*args, **kwargs)
```

```
print(f"[{self.prefix*} {func.__name__* 执行成功}")
```



```
return result
```

```
except Exception as e:
```

```
print(f"[{self.prefix*} {func.__name__*} 出错: {e*}")
```

raise

return wrapper


```
@Logger(prefix="API")
```



```
def get_user(user_id):
```

```
if user_id < 0:
```

```
raise ValueError("无效的用户ID")
```

```
return {"id": user_id, "name": "Alice"}
```


保留原函数信息（重要！）

```
from functools import wraps
```



```
def my_decorator(f):
```

`@wraps(f)` # 保留函数名、文档字符串等信息

```
def wrapper(*args, **kwargs):
```

"""包装函数"""

```
return f(*args, **kwargs)
```

return wrapper

@my_decorator

```
def example():
```

"""原始函数的文档"""

pass


```
print(example.__name__) # example (而不是wrapper)
```

```
print(example.__doc__)    # 原始函数的文档
```


□1.2 上下文管理器 (Context Managers)

方式一：基于类的上下文管理器

```
class DatabaseConnection:
```

```
    """数据库连接上下文管理器"""
```

```
    def __init__(self, connection_string):
        self.connection_string = connection_string
        self.conn = None
```

```
    def __enter__(self):
        print("打开数据库连接...")
        # 模拟建立连接
        self.conn = {"connected": True}
        return self.conn
```

```
    def __exit__(self, exc_type, exc_val, exc_tb):
        print("关闭数据库连接...")
        # 清理资源
        self.conn = None
        # 如果返回True, 会抑制异常
        if exc_type is not None:
            print(f"发生异常: {exc_val}")
            return False # 不抑制异常, 让异常继续传播
```

使用方式

```
with DatabaseConnection("postgresql://localhost/mydb") as conn:
```

```
    print(f"连接状态: {conn}")
```

```
    # 执行数据库操作...
```

自动调用 __exit__ 关闭连接

方式二：使用 contextlib.contextmanager

```
from contextlib import contextmanager
```

```
import time
```

```
@contextmanager
```

```
def timer_context(name="operation"):
```

```
    """计时上下文管理器"""
```

```
    start = time.time()
```

```
    print(f"[{name}] 开始...")
```

```
    yield # 暂停, 执行with块内的代码
```

```
    elapsed = time.time() - start
```

```
    print(f"[{name}] 完成, 耗时: {elapsed:.2f}秒")
```

```
@contextmanager
```

```
def temporary_file(content):
```

```
    """临时文件上下文管理器"""
```

```
    import tempfile
```

```
import os
```


创建临时文件

```
fd, path = tempfile.mkstemp()
```

try:

```
with os.fdopen(fd, 'w') as f:
```



```
f.write(content)
```

```
yield path # 返回文件路径
```

finally:

确保清理临时文件

```
os.unlink(path)
```


实际使用示例


```
with timer_context("数据处理"):
```

```
time.sleep(0.5)
```

```
data = [i ** 2 for i in range(1000)]
```



```
with temporary_file("Hello, World!") as file_path:
```

```
with open(file_path, 'r') as f:
```

```
content = f.read()
```

```
print(f"文件内容: {content*}")
```


□1.3 生成器 (Generators)

```

# 基础生成器
def count_up_to(n):
    """生成从0到n的数字"""
    current = 0
    while current <= n:
        yield current
        current += 1

# 使用生成器
counter = count_up_to(5)
print(next(counter)) # 0
print(next(counter)) # 1
for num in counter:
    print(num)        # 2, 3, 4, 5

# 无限生成器
import random

def random_number_generator(low=0, high=100):
    """无限生成随机数"""
    while True:
        yield random.randint(low, high)

# 限制使用次数
rand_gen = random_number_generator()
first_5_randoms = [next(rand_gen) for _ in range(5)]
print(first_5_randoms)

# 生成器表达式（类似列表推导式）
squares = (x ** 2 for x in range(10)) # 生成器对象，惰性求值
print(list(squares)) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# 实际案例：分页读取大文件
def read_in_chunks(file_path, chunk_size=1024*1024):
    """按块读取大文件"""
    with open(file_path, 'r', encoding='utf-8') as f:
        while True:
            chunk = f.read(chunk_size)
            if not chunk:
                break
            yield chunk

# 处理大日志文件

```

```
for chunk in read_in_chunks('large_log.txt', chunk_size=4096):
```

```
process_chunk(chunk) # 处理每个块
```


管道式处理（链式生成器）

```
def filter_lines(lines, keyword):
```


"""过滤包含关键词的行"""

```
for line in lines:
```

```
if keyword in line:
```

yield line


```
def transform_lines(lines):
```

""转换行格式""


```
for line in lines:
```

```
yield line.strip().upper()
```


组合使用

```
file_path = 'data.txt'
```

keyword = 'ERROR'


```
processed = transform_lines(
```

```
filter_lines(
```

```
read_in_chunks(file_path),
```

keyword

for line in processed:


```
print(line) # 输出包含ERROR的大写行
```

❑ 2. Python 异步编程 (1.5小时)

❑ 2.1 异步基础概念

```
import asyncio
import time

# 同步 vs 异步对比

# 同步版本（阻塞）
def sync_task(name, seconds):
    print(f"[同步] 任务 {name* 开始}")
    time.sleep(seconds) # 阻塞当前线程
    print(f"[同步] 任务 {name* 完成（耗时 {seconds*s}）")

# 异步版本（非阻塞）
async def async_task(name, seconds):
    print(f"[异步] 任务 {name* 开始}")
    await asyncio.sleep(seconds) # 挂起协程，不阻塞事件循环
    print(f"[异步] 任务 {name* 完成（耗时 {seconds*s}）")

# 执行同步任务（串行）
print("—— 同步执行 ——")
start = time.time()
sync_task('A', 2)
sync_task('B', 2)
sync_task('C', 2)
print(f"总耗时: {time.time() - start:.2f*s}") # 约6秒

# 执行异步任务（并发）
print("\n—— 异步执行 ——")
start = time.time()

async def main():
    # 并发运行多个任务
    await asyncio.gather(
        async_task('A', 2),
        async_task('B', 2),
        async_task('C', 2)
    )

asyncio.run(main())
print(f"总耗时: {time.time() - start:.2f*s}") # 约2秒!
```

❑ 2.2 async/await 语法详解

```

import aiohttp
import asyncio

async def fetch_data(url):
    """异步获取URL内容"""
    async with aiohttp.ClientSession() as session:
        async with session.get(url) as response:
            return await response.text()

async def process_data(data):
    """异步处理数据"""
    await asyncio.sleep(0.1) # 模拟IO密集型操作
    return len(data)

async def main():
    urls = [
        'https://api.example.com/users',
        'https://api.example.com/posts',
        'https://api.example.com/comments'
    ]

    # 方式1: 并发获取所有URL
    tasks = [fetch_data(url) for url in urls]
    results = await asyncio.gather(*tasks, return_exceptions=True)

    for url, result in zip(urls, results):
        if isinstance(result, Exception):
            print(f"L {url*: {result*}")
        else:
            print(f"□{url*: 获取到 {len(result)* 字符}")

    # 方式2: 顺序处理但非阻塞
    for url in urls:
        data = await fetch_data(url)
        count = await process_data(data)
        print(f"处理完成: {count*")

# 创建和运行任务
async def controlled_execution():
    """控制并发数量"""
    semaphore = asyncio.Semaphore(3) # 最多3个并发

    async def bounded_fetch(url):
        async with semaphore:
            return await fetch_data(url)

```

```
tasks = [bounded_fetch(url) for url in urls]
```

```
results = await asyncio.gather(*tasks)
```

return results

□2.3 异步迭代器和上下文管理器


```

# 异步迭代器
class AsyncCounter:
    """异步计数器"""

    def __init__(self, stop):
        self.stop = stop

    def __aiter__(self):
        return self

    async def __anext__(self):
        if self.current >= self.stop:
            raise StopAsyncIteration

        await asyncio.sleep(0.1) # 模拟异步操作
        result = self.current
        self.current += 1
        return result

# 使用异步for循环
async def use_async_iterator():
    counter = AsyncCounter(5)
    async for number in counter:
        print(f"计数: {number*}")

# 异步上下文管理器
class AsyncDatabaseConnection:
    """异步数据库连接"""

    async def __aenter__(self):
        print("异步打开数据库连接...")
        await asyncio.sleep(0.1) # 模拟异步连接
        self.connection = {"connected": True}
        return self.connection

    async def __aexit__(self, exc_type, exc_val, exc_tb):
        print("异步关闭数据库连接...")
        await asyncio.sleep(0.05) # 模拟异步关闭
        self.connection = None
        return False

# 使用方式
async def use_async_context():
    async with AsyncDatabaseConnection() as conn:
        print(f"连接状态: {conn*}")
        # 执行异步数据库操作...

```

❑3. FastAPI 框架入门 (1.5小时)

❑3.1 为什么选择FastAPI?

FastAPI的优势:

✓ 高性能 - 基于Starlette和Pydantic, 性能接近Node.js/Go ✓ 自动文档 - 自动生成交互式API文档 (Swagger UI) ✓ 类型提示 - 利用Python类型注解实现数据验证 ✓ 现代特性 - 原生支持async/await ✓ 易于学习 - 直观的API设计, 快速上手 ✓ 生态完善 - 支持OAuth、WebSocket、GraphQL等

性能对比:

框架	请求/秒 (越高越好)
FastAPI	~35,000
Flask	~8,000
Django	~12,000
Express (Node.js)	~25,000

❑3.2 第一个FastAPI应用

```

# main.py
from fastapi import FastAPI
from typing import Optional

app = FastAPI(
    title="AI Agent API",
    description="AI Agent后端服务",
    version="1.0.0"
)

@app.get("/")
async def root():
    """根路径 - 健康检查"""
    return {
        "message": "Welcome to AI Agent API",
        "status": "running",
        "version": "1.0.0"
    }
    *

@app.get("/items/{item_id}")
async def read_item(item_id: int, q: Optional[str] = None):
    """
    获取单个物品

    - **item_id**: 物品ID（必须为整数）
    - **q**: 可选的搜索查询字符串
    """
    item = {
        "id": item_id,
        "name": f"Item {item_id}",
        "description": f"This is item {item_id}"
    }
    *

    if q:
        item["query"] = q

    return item

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

启动并访问:

```
# 启动服务器
uvicorn main:app --reload

# 访问API文档（自动生成）
# http://localhost:8000/docs          # Swagger UI
# http://localhost:8000/redoc        # ReDoc文档
```

□ 3.3 请求参数与数据验证

```

from fastapi import FastAPI, Query, Path, Body, HTTPException
from pydantic import BaseModel, Field, EmailStr
from typing import List, Optional
from datetime import datetime
from enum import Enum

app = FastAPI()

# 枚举类型
class UserRole(str, Enum):
    ADMIN = "admin"
    EDITOR = "editor"
    VIEWER = "viewer"

# Pydantic模型（请求数据验证）
class UserBase(BaseModel):
    username: str = Field(..., min_length=3, max_length=20, pattern=r"^[a-zA-Z0-9_]+$")
    email: EmailStr
    full_name: Optional[str] = Field(None, max_length=50)
    role: UserRole = UserRole.VIEWER

class UserCreate(UserBase):
    password: str = Field(..., min_length=8, regex=r"^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{8,}$")

class UserResponse(BaseModel):
    id: int
    username: str
    email: str
    full_name: Optional[str]
    role: UserRole
    created_at: datetime

    class Config:
        from_attributes = True

class UserUpdate(BaseModel):
    email: Optional[EmailStr] = None
    full_name: Optional[str] = Field(None, max_length=50)
    role: Optional[UserRole] = None

# 路由定义

```



```
@app.post("/users/", response_model=UserResponse, status_code=201)
```

```
async def create_user(user: UserCreate):
```


||||

创建新用户

- ****username****: 用户名（3-20个字符，仅字母数字下划线）

- ****email****: 有效邮箱地址

- ****password****: 密码（至少8位，包含字母和数字）

||||

模拟保存到数据库


```
new_user = {
```

"id": 1,

```
**user.dict(),
```

```
"created_at": datetime.now()
```



```
return new_user
```



```
@app.get("/users/", response_model=List[UserResponse])
```

```
async def list_users()
```

```
skip: int = Query(0, ge=0, description="跳过的记录数"),
```

```
limit: int = Query(10, ge=1, le=100, description="返回的最大记录数"),
```

```
role: Optional[UserRole] = Query(None, description="按角色筛选")
```

):

||||

获取用户列表

- **skip**: 分页偏移量 (默认0)

- **limit**: 每页数量（默认10，最大100）

- **role**: 可选的角色筛选

||||

模拟查询数据库

```
users = []
```



```
return users[skip : skip + limit]
```



```
@app.get("/users/{user_id*}", response_model=UserResponse)
```

```
async def get_user(
```

```
user_id: int = Path(..., gt=0, description="用户ID（正整数）")
```

):

||||

根据ID获取用户详情

||||

模拟查询

```
if user_id != 1:
```

```
raise HTTPException(status_code=404, detail="用户不存在")
```



```
return {
```

```
"id": user_id,
```



```
"username": "alice",
```

"email": "alice@example.com",

"full_name": "Alice Johnson",

```
"role": UserRole.ADMIN,
```

```
"created_at": datetime.now()
```



```
@app.patch("/users/{user_id*}", response_model=UserResponse)
```

```
async def update_user(
```

user_id: int,

user_update: UserUpdate

):

||||

更新用户信息（部分更新）

||||

模拟更新

```
update_data = user_update.dict(exclude_unset=True)
```



```
return {
```

```
"id": user_id,
```

```
"username": "alice",
```

```
**update_data,
```

```
"created_at": datetime.now()
```



```
@app.delete("/users/{user_id}", status_code=204)
```

```
async def delete_user(user_id: int):
```

||||

删除用户

||||

模拟删除

```
return None
```

□3.4 依赖注入系统

```

from fastapi import Depends, HTTPException, Header
from functools import lru_cache
import jwt
from typing import Generator

# 配置管理
class Settings:
    SECRET_KEY: str = "your-secret-key-here"
    ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 30

@lru_cache()
def get_settings():
    return Settings()

# 数据库会话
def get_db() -> Generator:
    """获取数据库会话"""
    from sqlalchemy.orm import Session
    from database import SessionLocal

    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# OAuth2密码流
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/token")

async def get_current_user(
    token: str = Depends(oauth2_scheme),
    settings: Settings = Depends(get_settings)
):
    """验证JWT令牌并返回当前用户"""
    credentials_exception = HTTPException(
        status_code=401,
        detail="无法验证凭据",
        headers={"WWW-Authenticate": "Bearer"},
    )

    try:

```

```
payload = jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM
```

M])

```
user_id: str = payload.get("sub")
```

```
if user_id is None:
```



```
raise credentials_exception
```

```
except jwt.PyJWTError:
```

```
raise credentials_exception
```


从数据库获取用户

```
user = get_user_by_id(int(user_id))
```

```
if user is None:
```

```
raise credentials_exception
```


return user


```
async def get_current_active_user(current_user = Depends(get_current_user)):
```

"""获取当前活跃用户"""

```
if not current_user.is_active:
```

```
raise HTTPException(status_code=400, detail="用户已被禁用")
```



```
return current_user
```


使用依赖的路由

```
@app.get("/users/me", response_model=UserResponse)
```

```
async def read_users_me(current_user: User = Depends(get_current_active_user)):
```

"""获取当前登录用户信息"""

```
return current_user
```



```
@app.get("/admin/dashboard")
```

```
async def admin_dashboard()
```

```
current_user: User = Depends(get_current_active_user)
```

):

"""管理员仪表盘（需要管理员权限）"""

```
if current_user.role != UserRole.ADMIN:
```



```
raise HTTPException(status_code=403, detail="权限不足")
```



```
return {"message": f"欢迎管理员 {current_user.username}**"
```

4. RESTful API 设计原则 (1.5小时)

4.1 REST架构风格

核心原则：

- ☐ 资源导向 - 一切皆资源（URI标识资源）
- ☐ 统一接口 - 使用标准HTTP方法
- ☐ 无状态 - 服务器不保存客户端状态
- ☐ 分层系统 - 客户端不知道是直接还是代理服务器
- ☐ 可缓存 - 响应应标记是否可缓存

HTTP方法语义：

方法	幂等性	安全性	用途
GET	√ 是	√ 安全	获取资源
POST	! 否	! 不安全	创建资源
PUT	√ 是	! 不安全	整体替换资源
PATCH	! 否	! 不安全	部分更新资源
DELETE	√ 是	! 不安全	删除资源

4.2 URI设计最佳实践

□好的URI设计

使用名词复数表示资源集合

```
GET /api/v1/users      # 获取所有用户
GET /api/v1/users/123   # 获取特定用户
POST /api/v1/users      # 创建新用户
PUT /api/v1/users/123   # 更新用户（整体替换）
PATCH /api/v1/users/123 # 更新用户（部分字段）
DELETE /api/v1/users/123 # 删除用户
```

使用查询参数进行筛选、排序、分页

```
GET /api/v1/posts?status=published&sort=-created_at&page=1&size=20
GET /api/v1/products?category=electronics&min_price=100&max_price=1000
```

嵌套资源（层级不宜过深，建议不超过3层）

```
GET /api/v1/users/123/posts      # 获取用户的文章列表
GET /api/v1/users/123/posts/456  # 获取用户的特定文章
POST /api/v1/users/123/posts     # 为用户创建文章
```

□差的URI设计

```
GET /api/v1/getAllUsers      # 应该用 GET /users
POST /api/v1/createUser      # 应该用 POST /users
DELETE /api/v1/deleteUser?id=123 # 应该用 DELETE /users/123
GET /api/v1/users/get/123    # 冗余动词
/api/v1/user                  # 应该用复数 users
```

□4.3 统一的响应格式

```

# response_models.py
from pydantic import BaseModel
from typing import Any, Optional, Generic, TypeVar, List
from datetime import datetime

T = TypeVar('T')

class ResponseBase(BaseModel, Generic[T]):
    """统一响应基类"""
    code: int = 200
    message: str = "success"
    data: Optional[T] = None
    timestamp: datetime = datetime.now()

class PaginatedData(BaseModel, Generic[T]):
    """分页数据结构"""
    items: List[T]
    total: int
    page: int
    size: int
    pages: int

class ErrorResponse(BaseModel):
    """错误响应"""
    code: int
    message: str
    details: Optional[Any] = None
    timestamp: datetime = datetime.now()

# 使用示例
@app.get("/users/", response_model=ResponseBase[PaginatedData[UserResponse]])
async def list_users_paginated(
    page: int = Query(1, ge=1),
    size: int = Query(10, ge=1, le=100)
):
    """分页获取用户列表"""
    users, total = await user_service.get_paginated(page, size)

    paginated_data = PaginatedData(
        items=users,
        total=total,
        page=page,
        size=size,
        pages=(total + size - 1) // size
    )

```



```
return ResponseBase[PaginatedData[UserResponse]](
```


data=paginated_data,

message=f"获取成功，共 {total}* 条记录"

全局异常处理器

```
@app.exception_handler(HTTPException)
```

```
async def http_exception_handler(request, exc):
```



```
return JsonResponse(
```

```
status_code=exc.status_code,
```

```
content=ErrorResponse(
```

```
code=exc.status_code,
```

message=exc.detail

```
).dict()
```


□4.4 版本控制策略

```
# 方式1: URL路径版本控制 (推荐)
app_v1 = FastAPI(title="API v1")

@app_v1.get("/items/")
def get_items_v1():
    return {"version": "v1", "items": []}

app_v2 = FastAPI(title="API v2")

@app_v2.get("/items/")
def get_items_v2():
    return {"version": "v2", "items": [], "metadata": {}}

# 在main.py中挂载
from fastapi import FastAPI

app = FastAPI()

app.mount("/api/v1", app_v1)
app.mount("/api/v2", app_v2)

# 方式2: 请求头版本控制
@app.get("/items/")
async def get_items(
    request: Request,
    x_api_version: Optional[str] = Header(None)
):
    version = x_api_version or "v1"

    if version == "v2":
        return {"version": "v2", "items": [], "metadata": {}}

    return {"version": "v1", "items": []}
```

□5. 中间件与错误处理 (1小时)

□5.1 自定义中间件


```

from fastapi import Request, Response
from starlette.middleware.base import BaseHTTPMiddleware
import time
import logging

logger = logging.getLogger(__name__)

class LoggingMiddleware(BaseHTTPMiddleware):
    """请求日志中间件"""

    async def dispatch(self, request: Request, call_next):
        # 记录请求信息
        start_time = time.time()

        logger.info(f"{request.method} {request.url.path}")

        # 处理请求
        response: Response = await call_next(request)

        # 记录响应信息
        process_time = time.time() - start_time
        response.headers["X-Process-Time"] = str(process_time)

        logger.info(
            f"{response.status_code} "
            f"| 耗时: {process_time:.3f}s"
        )

        return response

class CORSMiddleware(BaseHTTPMiddleware):
    """跨域资源共享中间件"""

    def __init__(
        self,
        app,
        allow_origins: list = ["*"],
        allow_methods: list = ["*"],
        allow_headers: list = ["*"],
    ):
        super().__init__(app)
        self.allow_origins = allow_origins
        self.allow_methods = allow_methods
        self.allow_headers = allow_headers

    async def dispatch(self, request: Request, call_next):
        # 处理预检请求

```

```
if request.method == "OPTIONS":
```

```
response = Response()
```

```
response.headers["Access-Control-Allow-Origin"] = ", ".join(self.allow_ori
```

gins)

```
response.headers["Access-Control-Allow-Methods"] = ", ".join(self.allow_me
```

thods)

```
response.headers["Access-Control-Allow-Headers"] = ", ".join(self.allow_he
```


aders)

return response

正常请求

```
response = await call_next(request)
```


添加CORS头

```
response.headers["Access-Control-Allow-Origin"] = ", ".join(self.allow_origins
```



```
response.headers["Access-Control-Allow-Credentials"] = "true"
```


return response


```
class RateLimitMiddleware(BaseHTTPMiddleware):
```

"""限流中间件（简单实现）"""


```
def __init__(self, app, max_requests: int = 100, window_seconds: int = 60):
```

```
super().__init__(app)
```

```
self.max_requests = max_requests
```

```
self.window_seconds = window_seconds
```

```
self.requests = {*
```



```
async def dispatch(self, request: Request, call_next):
```



```
client_ip = request.client.host
```

```
current_time = time.time()
```


清理过期记录

```
self.requests = {
```

ip: times

```
for ip, times in self.requests.items()
```

```
if any(t > current_time - self.window_seconds for t in times)
```


检查请求频率

```
if client_ip in self.requests:
```

```
recent_requests = [
```

```
t for t in self.requests[client_ip]
```

```
if t > current_time - self.window_seconds
```



```
if len(recent_requests) >= self.max_requests:
```

```
return JsonResponse(
```

status_code=429,

content={

"code": 429,

"message": "请求过于频繁，请稍后再试",

```
"retry_after": self.window_seconds
```



```
recent_requests.append(current_time)
```

```
self.requests[client_ip] = recent_requests
```

else:

```
self.requests[client_ip] = [current_time]
```



```
return await call_next(request)
```


注册中间件

```
app.add_middleware(LoggingMiddleware)
```

```
app.add_middleware(CORSMiddleware)
```

```
app.add_middleware(RateLimitMiddleware, max_requests=60, window_seconds=60)
```

□5.2 全局异常处理


```

from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError
from pydantic import ValidationError

app = FastAPI()

class AppException(Exception):
    """自定义应用异常"""
    def __init__(self, code: int, message: str, details: Any = None):
        self.code = code
        self.message = message
        self.details = details

class NotFoundException(AppException):
    """资源未找到异常"""
    def __init__(self, resource: str, id: Any):
        super().__init__(
            code=404,
            message=f"{resource}* 未找到",
            details={"resource": resource, "id": id*
        )

class UnauthorizedException(AppException):
    """未授权异常"""
    def __init__(self, message: str = "未授权访问"):
        super().__init__(code=401, message=message)

class ForbiddenException(AppException):
    """禁止访问异常"""
    def __init__(self, message: str = "权限不足"):
        super().__init__(code=403, message=message)

class ValidationException(AppException):
    """数据验证异常"""
    def __init__(self, errors: list):
        super().__init__(
            code=422,
            message="数据验证失败",
            details=errors
        )

# 全局异常处理器

```

```
@app.exception_handler(AppException)
```

```
async def app_exception_handler(request: Request, exc: AppException):
```

```
return JsonResponse(
```

```
status_code=exc.code,
```

content={

```
"code": exc.code,
```

```
"message": exc.message,
```



```
"details": exc.details,
```

```
"path": request.url.path,
```

```
"timestamp": datetime.utcnow().isoformat()
```



```
@app.exception_handler(HTTPException)
```



```
async def http_exception_handler(request: Request, exc: HTTPException):
```

```
return JsonResponse(
```

```
status_code=exc.status_code,
```

content={

```
"code": exc.status_code,
```

```
"message": exc.detail,
```

```
"path": request.url.path,
```

```
"timestamp": datetime.utcnow().isoformat()
```



```
@app.exception_handler(RequestValidationError)
```

```
async def validation_exception_handler(request: Request, exc: RequestValidationError):
```

```
errors = []
```

```
for error in exc.errors():
```



```
errors.append({
```

```
"field": ".".join(str(loc) for loc in error["loc"]),
```

```
"message": error["msg"],
```

"type": error["type"]


```
return JsonResponse(
```

status_code=422,

content={

"code": 422,

"message": "请求参数验证失败",

"details": errors,

```
"path": request.url.path,
```

```
"timestamp": datetime.utcnow().isoformat()
```


使用自定义异常

```
@app.get("/products/{product_id*}")
```

```
async def get_product(product_id: int):
```

```
product = await product_service.get_by_id(product_id)
```


if not product:


```
raise NotFoundException("产品", product_id)
```


return product

□ 6. 数据库 ORM 集成 (1.5小时)

□ 6.1 SQLAlchemy 配置

```
# database.py
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from typing import Generator

DATABASE_URL = "postgresql://user:password@localhost/dbname"

engine = create_engine(DATABASE_URL, pool_size=20, max_overflow=10)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db() -> Generator:
    """依赖项: 获取数据库会话"""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

□ 6.2 定义数据模型

```

# models.py
from sqlalchemy import Column, Integer, String, Boolean, DateTime, Text, ForeignKey
from sqlalchemy.orm import relationship
from datetime import datetime
from database import Base

class TimestampMixin:
    """时间戳混入类"""
    created_at = Column(DateTime, default=datetime.utcnow, nullable=False)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

class User(Base, TimestampMixin):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String(50), unique=True, index=True, nullable=False)
    email = Column(String(255), unique=True, index=True, nullable=False)
    hashed_password = Column(String(255), nullable=False)
    full_name = Column(String(100))
    is_active = Column(Boolean, default=True)
    is_admin = Column(Boolean, default=False)

    # 关系
    posts = relationship("Post", back_populates="author")
    comments = relationship("Comment", back_populates="author")

class Post(Base, TimestampMixin):
    __tablename__ = "posts"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(200), nullable=False)
    content = Column(Text, nullable=False)
    published = Column(Boolean, default=False)
    view_count = Column(Integer, default=0)

    # 外键
    author_id = Column(Integer, ForeignKey("users.id"), nullable=False)

    # 关系
    author = relationship("User", back_populates="posts")
    comments = relationship("Comment", back_populates="post")

class Comment(Base, TimestampMixin):
    __tablename__ = "comments"

```

```
id = Column(Integer, primary_key=True, index=True)
```

```
content = Column(Text, nullable=False)
```


外键

```
post_id = Column(Integer, ForeignKey("posts.id"), nullable=False)
```

```
author_id = Column(Integer, ForeignKey("users.id"), nullable=False)
```


关系

```
post = relationship("Post", back_populates="comments")
```

```
author = relationship("User", back_populates="comments")
```

□ 6.3 Pydantic Schema (序列化)


```
# schemas.py
from pydantic import BaseModel, EmailStr
from typing import Optional, List
from datetime import datetime

class UserBase(BaseModel):
    email: EmailStr
    username: str
    full_name: Optional[str] = None

class UserCreate(UserBase):
    password: str

class UserUpdate(BaseModel):
    email: Optional[EmailStr] = None
    username: Optional[str] = None
    full_name: Optional[str] = None
    is_active: Optional[bool] = None

class UserInDB(UserBase):
    id: int
    hashed_password: str
    is_active: bool
    is_admin: bool
    created_at: datetime
    updated_at: datetime

    class Config:
        from_attributes = True

class UserResponse(BaseModel):
    id: int
    email: EmailStr
    username: str
    full_name: Optional[str]
    is_active: bool
    created_at: datetime

    class Config:
        from_attributes = True

class PostBase(BaseModel):
    title: str
```

content: str

```
published: bool = False
```



```
class PostCreate(PostBase):
```

pass


```
class PostUpdate(BaseModel):
```

```
title: Optional[str] = None
```

```
content: Optional[str] = None
```

```
published: Optional[bool] = None
```



```
class PostResponse(PostBase):
```


id: int

author_id: int

view_count: int

created_at: datetime

updated_at: datetime

```
author: Optional[UserResponse] = None
```



```
class Config:
```



```
from_attributes = True
```



```
class PostWithComments(PostResponse):
```

```
comments: List['CommentResponse'] = []
```



```
class CommentBase(BaseModel):
```


content: str


```
class CommentCreate(CommentBase):
```

post_id: int


```
class CommentResponse(CommentBase):
```


id: int

post_id: int

author_id: int

created_at: datetime

```
author: Optional[UserResponse] = None
```



```
class Config:
```

```
from_attributes = True
```


□ 6.4 CRUD 操作实现

```

# crud.py
from sqlalchemy.orm import Session
from models import User, Post, Comment
from schemas import UserCreate, UserUpdate, PostCreate, PostUpdate, CommentCreate
from security import get_password_hash, verify_password

# ===== 用户 CRUD =====

def get_user(db: Session, user_id: int):
    """根据ID获取用户"""
    return db.query(User).filter(User.id == user_id).first()

def get_user_by_email(db: Session, email: str):
    """根据邮箱获取用户"""
    return db.query(User).filter(User.email == email).first()

def get_user_by_username(db: Session, username: str):
    """根据用户名获取用户"""
    return db.query(User).filter(User.username == username).first()

def get_users(db: Session, skip: int = 0, limit: int = 100):
    """分页获取用户列表"""
    return db.query(User).offset(skip).limit(limit).all()

def create_user(db: Session, user: UserCreate):
    """创建用户"""
    hashed_password = get_password_hash(user.password)
    db_user = User(
        email=user.email,
        username=user.username,
        hashed_password=hashed_password,
        full_name=user.full_name
    )
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user

def update_user(db: Session, user_id: int, user_update: UserUpdate):
    """更新用户信息"""
    db_user = get_user(db, user_id)
    if not db_user:
        return None

```



```
update_data = user_update.model_dump(exclude_unset=True)
```

```
for field, value in update_data.items():
```

```
setattr(db_user, field, value)
```



```
db.commit()
```



```
db.refresh(db_user)
```

```
return db_user
```



```
def delete_user(db: Session, user_id: int):
```

"""删除用户"""

```
db_user = get_user(db, user_id)
```

```
if not db_user:
```



```
return False
```



```
db.delete(db_user)
```

```
db.commit()
```

```
return True
```



```
def authenticate_user(db: Session, username: str, password: str):
```


""验证用户凭据""

```
user = get_user_by_username(db, username)
```

```
if not user:
```

```
return False
```

```
if not verify_password(password, user.hashed_password):
```

```
return False
```

return user

===== 文章 CRUD =====


```
def get_post(db: Session, post_id: int):
```

"""根据ID获取文章"""

```
return db.query(Post).filter(Post.id == post_id).first()
```



```
def get_posts(
```

db: Session,

```
skip: int = 0,
```

```
limit: int = 100,
```

```
published_only: bool = False
```

):

"""分页获取文章列表"""

```
query = db.query(Post)
```



```
if published_only:
```

```
query = query.filter(Post.published == True)
```

```
return query.offset(skip).limit(limit).all()
```



```
def create_post(db: Session, post: PostCreate, author_id: int):
```

""创建文章""

```
db_post = Post(**post.model_dump(), author_id=author_id)
```



```
db.add(db_post)
```

```
db.commit()
```

```
db.refresh(db_post)
```

```
return db_post
```



```
def update_post(db: Session, post_id: int, post_update: PostUpdate):
```

""更新文章""


```
db_post = get_post(db, post_id)
```

```
if not db_post:
```

```
return None
```



```
update_data = post_update.model_dump(exclude_unset=True)
```

```
for field, value in update_data.items():
```

```
setattr(db_post, field, value)
```



```
db.commit()
```

```
db.refresh(db_post)
```

```
return db_post
```



```
def increment_view_count(db: Session, post_id: int):
```

""增加浏览次数""

```
db_post = get_post(db, post_id)
```



```
if db_post:
```

```
db_post.view_count += 1
```

```
db.commit()
```

```
db.refresh(db_post)
```

```
return db_post
```

□6.5 完整路由整合

```

# routers/users.py
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from typing import List
from dependencies import get_db, get_current_active_user
from crud import get_user, get_users, create_user, update_user, delete_user
from schemas import UserCreate, UserUpdate, UserResponse
from models import User

router = APIRouter(prefix="/users", tags=["Users"])

@router.post("/", response_model=UserResponse, status_code=status.HTTP_201_CREATED)
async def create_new_user(
    user: UserCreate,
    db: Session = Depends(get_db)
):
    """注册新用户"""
    # 检查用户名和邮箱是否已存在
    db_user = get_user_by_username(db, user.username)
    if db_user:
        raise HTTPException(status_code=400, detail="用户名已存在")

    db_email = get_user_by_email(db, user.email)
    if db_email:
        raise HTTPException(status_code=400, detail="邮箱已被注册")

    return create_user(db=db, user=user)

@router.get("/", response_model=List[UserResponse])
async def read_users(
    skip: int = 0,
    limit: int = 100,
    db: Session = Depends(get_db)
):
    """获取用户列表"""
    users = get_users(db, skip=skip, limit=limit)
    return users

@router.get("/me", response_model=UserResponse)
async def read_users_me(
    current_user: User = Depends(get_current_active_user)
):
    """获取当前用户信息"""
    return current_user

```

```
@router.get("/{user_id*}", response_model=UserResponse)
```



```
async def read_user(
```

user_id: int,

```
db: Session = Depends(get_db)
```

):

"""根据ID获取用户"""

```
db_user = get_user(db, user_id=user_id)
```

```
if db_user is None:
```

```
raise HTTPException(status_code=404, detail="用户不存在")
```



```
return db_user
```



```
@router.put("/{user_id*", response_model=UserResponse)
```

```
async def update_existing_user(
```

user_id: int,

user_update: UserUpdate,

```
db: Session = Depends(get_db),
```



```
current_user: User = Depends(get_current_active_user)
```

):

""更新用户信息（只能更新自己或管理员）""

```
if current_user.id != user_id and not current_user.is_admin:
```

```
raise HTTPException(status_code=403, detail="只能修改自己的信息")
```



```
db_user = update_user(db, user_id=user_id, user_update=user_update)
```

```
if db_user is None:
```



```
raise HTTPException(status_code=404, detail="用户不存在")
```

```
return db_user
```



```
@router.delete("/{user_id*", status_code=status.HTTP_204_NO_CONTENT)
```

```
async def delete_existing_user(
```

user_id: int,

```
db: Session = Depends(get_db),
```



```
current_user: User = Depends(get_current_active_user)
```

):

"""删除用户"""

```
if current_user.id != user_id and not current_user.is_admin:
```

```
raise HTTPException(status_code=403, status_code=403, deta
```

il="只能删除自己的账号")


```
success = delete_user(db, user_id=user_id)
```


if not success:

```
raise HTTPException(status_code=404, detail="用户不存在")
```

```
return None
```

*> 实践任务清单

☐ 任务1：构建完整的博客API（3小时）

功能需求：

✓ 用户认证

- ☐ 注册（用户名/邮箱/密码）
- ☐ 登录（返回JWT令牌）
- ☐ 获取当前用户信息

✓ 文章管理

- ☐ 创建文章（标题/内容/发布状态）
- ☐ 获取文章列表（支持分页、筛选）
- ☐ 获取文章详情（增加浏览量）
- ☐ 更新文章
- ☐ 删除文章

✓ 评论系统

- ☐ 为文章添加评论
- ☐ 获取文章的评论列表

技术要求：

- ☐ 使用FastAPI框架
- ☐ PostgreSQL数据库 + SQLAlchemy ORM
- ☐ Pydantic数据验证
- ☐ JWT身份认证
- ☐ 统一的错误处理和响应格式
- ☐ Swagger自动文档

项目结构：

```
blog-api/
+-- main.py           # 应用入口
+-- config.py         # 配置文件
+-- database.py       # 数据库连接
+-- models/          # SQLAlchemy模型
|   +-- __init__.py
|   +-- user.py
|   +-- post.py
|   +-- comment.py
+-- schemas/          # Pydantic模型
|   +-- __init__.py
|   +-- user.py
|   +-- post.py
|   +-- comment.py
+-- crud/             # 数据库操作
|   +-- __init__.py
|   +-- user.py
|   +-- post.py
|   +-- comment.py
+-- routers/          # 路由
|   +-- __init__.py
|   +-- auth.py
|   +-- users.py
|   +-- posts.py
|   +-- comments.py
+-- dependencies.py   # 依赖注入
+-- security.py       # 安全相关（密码哈希、JWT）
+-- requirements.txt
```

检验标准：

- ☐ [] 所有API端点能正常工作（通过Swagger UI测试）
- ☐ [] 用户注册和登录流程完整
- ☐ [] 文章CRUD操作正确
- ☐ [] 评论功能正常
- ☐ [] 分页查询有效
- ☐ [] 错误处理友好且统一
- ☐ [] API文档清晰完整

☐任务2：编写单元测试（1.5小时）

测试要求：

- ☐ 测试所有CRUD接口
- ☐ 测试认证流程
- ☐ 测试边界情况（非法输入、权限不足等）
- ☐ 使用pytest + httpx（异步测试客户端）

示例代码：

```

# tests/test_posts.py
import pytest
from httpx import AsyncClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from database import Base, get_db
from main import app

TEST_DATABASE_URL = "postgresql+asyncpg://test:test@localhost/test_db"

engine = create_async_engine(TEST_DATABASE_URL)
TestingSessionLocal = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False
)

@pytest.fixture
async def db_session():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

    async with TestingSessionLocal() as session:
        yield session

    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.drop_all)

@pytest.fixture
async def client(db_session):
    async def override_get_db():
        yield db_session

    app.dependency_overrides[get_db] = override_get_db

    async with AsyncClient(app=app, base_url="http://test") as ac:
        yield ac

    app.dependency_overrides.clear()

@pytest.mark.asyncio
async def test_create_post(client, db_session):
    """测试创建文章"""
    # 先创建用户
    user_data = {
        "email": "test@example.com",
        "username": "testuser",
        "password": "testpass123",

```

"full_name": "Test User"


```
response = await client.post("/users/", json=user_data)
```

```
assert response.status_code == 201
```

```
user = response.json()
```


登录获取token

```
login_data = {"username": "testuser", "password": "testpass123"}
```



```
response = await client.post("/token", data=login_data)
```

```
assert response.status_code == 200
```

```
token = response.json()["access_token"]
```

```
headers = {"Authorization": f"Bearer {token}"}
```


创建文章

```
post_data = {
```

"title": "Test Post",

"content": "This is test content",

"published": True


```
response = await client.post("/posts/", json=post_data, headers=headers)
```

```
assert response.status_code == 201
```

```
post = response.json()
```



```
assert post["title"] == "Test Post"
```

```
assert post["author_id"] == user["id"]
```

```
assert post["published"] is True
```


`@pytest.mark.asyncio`

```
async def test_get_posts_pagination(client, db_session):
```

"""测试分页获取文章"""


```
response = await client.get("/posts/?page=1&size=10")
```

```
assert response.status_code == 200
```



```
data = response.json()
```

```
assert "items" in data["data"]
```

```
assert "total" in data["data"]
```

```
assert "page" in data["data"]
```

```
assert "pages" in data["data"]
```



```
if __name__ == "__main__":
```

```
pytest.main(["-v"])
```

运行测试：

```
# 安装测试依赖
pip install pytest pytest-asyncio httpx asyncpg

# 运行测试
pytest tests/ -v

# 查看覆盖率
pytest tests/ --cov=app --cov-report=html
```

检验标准：

- ☐ [] 所有测试通过
- ☐ [] 测试覆盖主要功能和边界情况
- ☐ [] 代码覆盖率 > 80%
- ☐ [] 测试运行稳定可靠

** 推荐学习资源

☐ 官方文档

- ☐ FastAPI 官方文档 - 最权威的学习资料
- ☐ Pydantic 文档 - 数据验证库
- ☐ SQLAlchemy 文档 - ORM框架
- ☐ Python asyncio 文档 - 异步编程

☐ 推荐教程

- ☐ FastAPI 实战视频 (B站/YouTube) - 从零构建完整项目
- ☐ RealWorld 示例项目 - 符合真实世界规范的FastAPI应用

☐ GitHub项目

- ☐ fastapi-realworld-example-app - RealWorld后端实现
- ☐ full-stack-fastapi-template - Tiangolo官方模板
- ☐ fastapi-api-utils - FastAPI工具集

☐ 最佳实践

- ☐ FastAPI 最佳实践指南 - 生产环境经验总结
 - ☐ Production-ready API - API设计规范
-

✓ 今日自测题

☐ 选择题（每题10分，共100分）

- ☐ 以下哪个不是Python装饰器的典型用途？
 - ☐ A. 日志记录
 - ☐ B. 性能计时
 - ☐ C. 权限验证
 - ☐ D. 内存管理 !! 这是垃圾回收器的工作
- ☐ `async` 和 `await` 的主要作用？
 - ☐ A. 提高代码执行速度
 - ☐ B. 实现非阻塞的异步IO操作
 - ☐ C. 自动并行化代码
 - ☐ D. 替代多线程
- ☐ FastAPI中`Depends()` 的作用？
 - ☐ A. 定义路由
 - ☐ B. 实现依赖注入
 - ☐ C. 数据验证
 - ☐ D. 错误处理
- ☐ RESTful API中，PUT和PATCH的区别？
 - ☐ A. 没有区别
 - ☐ B. PUT用于创建，PATCH用于更新
 - ☐ C. PUT整体替换资源，PATCH部分更新
 - ☐ D. PATCH比PUT更安全
- ☐ Pydantic的主要作用？
 - ☐ A. Web框架
 - ☐ B. 数据验证和序列化
 - ☐ C. 数据库ORM
 - ☐ D. 任务队列
- ☐ SQLAlchemy ORM中`relationship()` 的作用？
 - ☐ A. 定义表之间的关系
 - ☐ B. 创建索引
 - ☐ C. 设置主键
 - ☐ D. 定义外键约束
- ☐ HTTP状态码 422 表示什么？

- ☐ A. 未找到资源
 - ☐ B. 服务器内部错误
 - ☐ C. 无法处理的实体（验证失败）
 - ☐ D. 未授权
- ☐ yield 在函数中的作用？
- ☐ A. 返回值并结束函数
 - ☐ B. 创建生成器，暂停执行并返回值
 - ☐ C. 导入模块
 - ☐ D. 定义协程
- ☐ FastAPI中间件的执行顺序？
- ☐ A. 按照添加顺序执行
 - ☐ B. 反向执行
 - ☐ C. 随机执行
 - ☐ D. 并行执行
- ☐ 以下哪个不是好的API设计实践？
- ☐ A. 使用名词复数作为资源名
 - ☐ B. 返回统一的响应格式
 - ☐ C. 在URL中包含动词（如/getUsers）!! 这违反了REST规范
 - ☐ D. 合理使用HTTP状态码

☐答案与解析

- ☐ 答案：D - 装饰器用于增强函数行为，内存管理由Python解释器的垃圾回收机制负责。
- ☐ 答案：B - async/await用于编写异步代码，主要解决IO阻塞问题，提高并发能力。
- ☐ 答案：B - Depends()是FastAPI的依赖注入系统，用于共享和管理组件（如数据库会话、认证等）。
- ☐ 答案：C - PUT要求提供完整的资源表示进行替换，PATCH只提交需要修改的字段。
- ☐ 答案：B - Pydantic是一个强大的数据验证库，使用Python类型注解进行数据校验。
- ☐ 答案：A - relationship()定义ORM模型之间的关联关系（一对多、多对多等）。
- ☐ 答案：C - 422 Unprocessable Entity通常表示请求格式正确但语义错误（如数据验证失败）。
- ☐ 答案：B - 包含yield的函数成为生成器，可以暂停执行并在下次调用时恢复。
- ☐ 答案：A - 中间件按照add_middleware()的添加顺序执行（LIFO模式对于响应处理）。
- ☐ 答案：C - RESTful API应该使用HTTP方法表达动作，而不是在URL中使用动词。

评分标准：

- ☐ 90-100分：√>> 优秀！FastAPI掌握扎实！
 - ☐ 70-89分：*☐良好！建议复习错题知识点
 - ☐ 60-69分：!! 及格！需要加强练习
 - ☐ 60分以下：*☐建议重新学习今日内容
-

** 今日总结

☐ 关键收获

- ☐ _____
- ☐ _____
- ☐ _____

☐ 遇到的挑战

- ☐ 问题：解决：
- ☐ 问题：解决：

☐ 明日预告

Day 4: 数据库技术应用 将涵盖：

- ☐ PostgreSQL高级特性和优化技巧
- ☐ Redis数据结构和应用场景
- ☐ 数据库设计和规范化
- ☐ 缓存策略与会话管理
- ☐ 性能调优和监控

准备工作：

- ☐ [] 安装PostgreSQL和Redis（或使用Docker）
- ☐ [] 复习SQL基础知识
- ☐ [] 准备测试数据集

** 实战项目：BlogAPI 博客系统后端

☐ 项目概览

项目名称: Blog API - RESTful博客系统后端 路径:03-后端开发技能提升/blog-api/ 完成度: ✓
100% 文件数: 14个核心文件 代码量: 1500+ 行 技术栈: FastAPI + SQLAlchemy + PostgreSQL + JWT
认证 + Pydantic

☐ 核心特性

✓ 用户系统 - 注册/登录/JWT认证/用户CRUD ✓ 文章管理 - 创建/编辑/删除/分页查询 ✓ 数据库
ORM - SQLAlchemy模型定义 + 关系映射 ✓ 数据验证 - Pydantic Schema请求/响应验证 ✓ 统一错误
处理 - 全局异常处理器 + 自定义异常类 ✓ Swagger文档 - 自动生成交互式API文档

☐ 项目架构


```
blog-api/
+-- app/
|   +-- main.py           # FastAPI应用入口（CORS/Lifespan/Router
s注册）
|   +-- config.py         # Pydantic Settings配置
|   +-- database.py       # SQLAlchemy引擎 + 会话工厂
|   |
|   +-- models/           # ORM模型
|   |   +-- user.py       # User模型（id/email/username/password
/timestamps）
|   |
|   +-- schemas/          # Pydantic数据模型
|   |   +-- user.py       # UserCreate/UserLogin/UserUpdate/UserResponse/Token
|   |
|   +-- routers/          # 路由
|   |   +-- users.py      # 7个RESTful用户接口
|   |       +-- POST /register    # 用户注册
|   |       +-- POST /login      # 用户登录（返回JWT）
|   |       +-- GET /users       # 用户列表
|   |       +-- GET /users/me    # 当前用户信息
|   |       +-- GET /users/{id*} # 用户详情
|   |       +-- PUT /users/{id*} # 更新用户
|   |       +-- DELETE /users/{id*} # 删除用户
|
+-- tests/
|   +-- test_users.py      # API测试脚本
|
+-- requirements.txt       # Python依赖
+-- README.md              # 项目说明
```

□核心API端点

方法	路径	功能	认证
POST	/api/register	用户注册	!
POST	/api/login	登录获取JWT	!
GET	/api/users/me	获取当前用户	✓ JWT
GET	/api/users/{id}	获取用户详情	✓ JWT
PUT	/api/users/{id}	更新用户信息	✓ JWT
DELETE	/api/users/{id}	删除用户	✓ JWT (Admin)

□技术亮点

❑1. SQLAlchemy连接池配置 (database.py)

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

engine = create_engine(
    DATABASE_URL,
    pool_size=10,          # 连接池大小
    max_overflow=20,       # 溢出连接数
    pool_recycle=3600,     # 回收时间（秒）
    pool_pre_ping=True    # 连接前预检
)

SessionLocal = sessionmaker(bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

❑2. Pydantic数据验证 (schemas/user.py)

```
from pydantic import BaseModel, EmailStr, Field

class UserCreate(BaseModel):
    email: EmailStr          # 邮箱格式验证
    username: str = Field(..., min_length=3, max_length=20) # 长度限制
    password: str = Field(..., regex=r'^(?=.*[A-Za-z])(?=.*\d)') # 正则强度

class UserResponse(BaseModel):
    id: int
    email: str
    username: str
    is_active: bool
    created_at: datetime

    class Config:
        from_attributes = True # 支持ORM对象转换
```

❑3. JWT认证流程 (routers/users.py)

```
from fastapi.security import OAuth2PasswordBearer
from jose import jwt, JWTError

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/login")

async def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id: str = payload.get("sub")
        if user_id is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception

    user = get_user_by_id(int(user_id))
    if user is None:
        raise credentials_exception
    return user
```

❑快速启动

```
# 1. 进入项目目录
cd 03-后端开发技能提升/blog-api

# 2. 创建虚拟环境
python -m venv venv
venv\Scripts\activate # Windows
# source venv/bin/activate # Linux/macOS

# 3. 安装依赖
pip install -r requirements.txt
# 主要依赖: fastapi, uvicorn, sqlalchemy, pydantic, python-jose, bcrypt,
# psycpg2-binary

# 4. 配置数据库
# 确保PostgreSQL已启动并创建了数据库
set DATABASE_URL=postgresql://user:password@localhost:5432/blogdb # Windows

# 5. 初始化数据库表
alembic upgrade head # 或手动执行SQL创建表

# 6. 启动服务
uvicorn app.main:app --reload --port 8000

# 7. 访问API文档
# http://localhost:8000/docs (Swagger UI)
# http://localhost:8000/redoc (ReDoc文档)

# 8. 运行测试
pytest tests/test_users.py -v
```

□测试示例

```
# 运行用户API测试
cd blog-api
python tests/test_users.py

# 测试输出示例:
# □健康检查通过: {"status":"ok"}
# □用户创建成功: {"email":"test@example.com", "username":"testuser", ...}
# □用户列表获取成功: [...]
# □总共测试 5 个用例, 通过 5 个 □
```

□验收标准

- ☐ [] 所有7个API端点能正常工作（通过Swagger UI测试）
- ☐ [] 用户注册和登录流程完整（密码哈希存储）
- ☐ [] JWT令牌认证正常（过期时间30分钟）
- ☐ [] 数据库CRUD操作正确（增删改查）
- ☐ [] Pydantic数据验证生效（非法输入被拒绝）
- ☐ [] 错误处理友好且统一（400/401/403/404/422/500）
- ☐ [] API文档清晰完整（Swagger自动生成）
- ☐ [] 单元测试全部通过

*📄模块导航

Day 2: 前端技术复习与强化 | Day 4: 数据库技术应用 | ✓📄返回课程首页

✓* Day 3 完成！你已具备构建生产级后端API的能力！

明日将深入数据库世界，打造高性能的数据存储层！